

COGNOME:

NOME:

MATRICOLA:

Appello di Algoritmi e Strutture Dati (9CFU), 29 Agosto 2023.

Parte I - Usare voto prova parziale - SI () NO ()

Esercizio 1 [3 punti]Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

- (A) $\sqrt{n}$, $n^2 \log n$, $2^{\sqrt{n}}$, $2^{\log n^2}$, 10^n
- (B) $\sqrt{n}$, $(\log n)^4$, $2^{\log n^2}$, $n^{\log n}$, 10^n
- ~~(C)~~ $\log n$, $2^{\log n^2}$, $\binom{n}{2}$, $n^{\log n}$, $2^{\sqrt{n} \log n}$
- (D) Nessuna delle precedenti

Esercizio 2 [4 punti]Si considerino l'algoritmo CRUEL (vecchia conoscenza dell'esame di ASD) e la sua subroutine UNUSUAL, entrambi descritti di seguito. L'input è un array $A[p..q]$ di interi di dimensione n , con n potenza di 2.**Algorithm** CRUEL(A, p, q)

```

1: if  $q - p + 1 > 1$  then
2:   CRUEL( $A, p, (q + p - 1)/2$ )
3:   CRUEL( $A, (q + p - 1)/2 + 1, q$ )
4:   UNUSUAL( $A, p, q$ )
5: end if

```

$$T(n) = 2T(n/2) + \Theta(n^{\log_2 3})$$

$$a=2 \quad b=2 \quad n^{\log_2 2} = n < n^{\log_2 3}$$

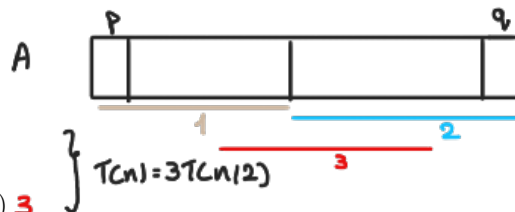
$$T(n) = \Theta(n^{\log_2 3})$$

Algorithm UNUSUAL(A, p, q)

```

1: if  $q - p + 1 = 2$  then
2:   if  $A[p] > A[p+1]$  then
3:     temp =  $A[p+1]$ 
4:      $A[p+1] = A[p]$ 
5:      $A[p] = temp$ 
6:   end if
7: else
8:   for  $i \leftarrow p$  to  $(q + 3p - 3)/4$  do
9:     temp =  $A[i + (q - p + 1)/2]$ 
10:     $A[i + (q - p + 1)/2] = A[i + (q - p + 1)/4]$ 
11:     $A[i + (q - p + 1)/4] = temp$ 
12:  end for
13:  UNUSUAL( $A, p, (q + p - 1)/2$ )
14:  UNUSUAL( $A, (q + p - 1)/2 + 1, q$ )
15:  UNUSUAL( $A, (q + 3p - 3)/4 + 1, (3q + p - 1)/4$ )
16: end if

```



$$T(n) = 3T(n/2) + \Theta(n)$$

$$a=3 \quad b=2 \quad n^{\log_2 3} = n^{1.58} > n$$

$$T(n) = \Theta(n^{\log_2 3})$$

La complessità temporale nel caso peggiore dell'algoritmo CRUEL è:

- (A) $\Theta(n^2)$
- (B) Nessuna delle precedenti
- (C) $\Theta(n)$
- ~~(D)~~ $\Theta(n^{\log 3})$

Esercizio 3 [3 punti]

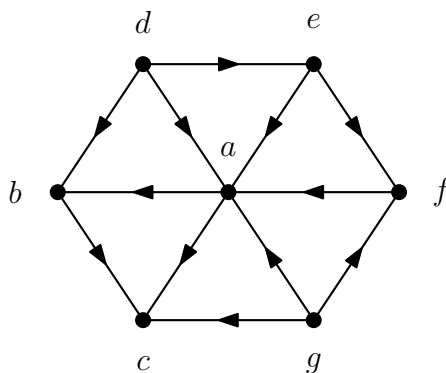
Indicare quale delle seguenti affermazioni è vera:

- ☒ (A) E' possibile ordinare un array di n interi con MERGE-SORT in $O(n^2)$ passi.
- (B) Qualsiasi algoritmo di ordinamento richiede $\Omega(n \log n)$ confronti nel caso peggiore.
- (C) Nessuna delle precedenti.
- (D) RADIX SORT usa COUNTING SORT come subroutine ma, sostituendo quest'ultimo con un qualsiasi algoritmo di ordinamento, la correttezza di RADIX SORT si mantiene.

Parte II

Esercizio 4 [5 punti]

Si esegua una visita in profondità del seguente grafo, supponendo che i vertici siano inizialmente ordinati alfabeticamente e che ogni lista di adiacenza sia ordinata alfabeticamente. Allora l'insieme degli archi in avanti è:



- (A) $\{(a, c), (d, a)\}$
- (B) Nessuno dei precedenti
- ☒ (C) $\{(a, c)\}$
- (D) $\{(d, b), (a, c)\}$

Esercizio 5 [5 punti]

Indicare quale delle seguenti affermazioni è falsa:

- (A) DFS può essere utilizzato per testare se un grafo diretto contiene un ciclo.
- ☒ (B) Qualsiasi grafo diretto aciclico ammette un unico ordinamento topologico.
- (C) Sia $G = (V, E)$ un grafo diretto aciclico con funzione peso $w: E \rightarrow \mathbb{R}$ e sia $s \in V$. E' possibile ottenere un cammino di peso minimo da s ad un qualsiasi vertice raggiungibile da s in $O(|V|^2)$.
- (D) Sia l'algoritmo di Bellman-Ford che l'algoritmo di Dijkstra usano come subroutine il rilassamento di un arco.

SBARRAMENTO - 12 punti sugli esercizi precedenti

Esercizio 6 [12 punti]

- Si consideri l'algoritmo di Dijkstra. Se ne descriva brevemente l'idea, si fornisca lo pseudo codice e infine si spieghi da cosa dipende il tempo di esecuzione. [8 punti]
- Fornire un esempio di grafo diretto e pesato $G = (V, E)$, con $|V| = |E| = 3$ e vertice sorgente s , sul quale l'algoritmo di Dijkstra non opera correttamente, cioè non determina correttamente un cammino minimo da s ad un qualsiasi vertice raggiungibile da s . Si disegni un tale G indicando chiaramente quali siano i pesi e quale il vertice sorgente s . [4 punti]

Esercizio 1 [3 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

(A) $\sqrt{n}, n^2 \log n, 2^{\sqrt{n}}, 2^{\log n^2}, 10^n$

(B) $\sqrt{n}, (\log n)^4, 2^{\log n^2}, n^{\log n}, 10^n$

(C) $\log n, 2^{\log n^2}, \binom{n}{2}, n^{\log n}, 2^{\sqrt{n} \log n}$

(D) Nessuna delle precedenti

A. $\sqrt{n} \leq n^2 \log n \leq 2^{\sqrt{n}} > 2^{\log n^2}$

• $\sqrt{n} = n^{1/2}$

• $2^{\sqrt{n}} = 2^{n^{1/2}}$

• $2^{\log n^2} = 2^{2 \log n} = n^2$

applico il log: $\log_2 2^{\sqrt{n}} = \sqrt{n} \log 2$
 $\log_2 n^2 = 2 \log n$
 $\sqrt{n} > \log n$

B. $\sqrt{n} > (\log n)^4$

C. $\log n \leq 2^{\log n^2} \leq \binom{n}{2} \leq n^{\log n} \leq n^{\sqrt{n}}$

• $2^{\log n^2} = n^2$

• $\binom{n}{2} \approx n^2$

• $2^{\sqrt{n} \log n} = n^{\sqrt{n}}$

1. Si consideri l'algoritmo di Dijkstra. Se ne descriva brevemente l'idea, si fornisca lo pseudo codice e infine si spieghi da cosa dipende il tempo di esecuzione. [8 punti]

Dijkstra: è un algoritmo greedy utilizzato per risolvere il problema dei cammini minimi da sorgente singola su un grafo pesato e orientato (o non orientato), con peso positivo (>0)

Il funzionamento:

1. Viene mantenuta una coda di min-priorità Q contenente tutti i vertici del grafo, dove la chiave di ciascun vertice è la sua stima corrente del cammino minimo (v.d.)
2. Ad ogni passo, l'algoritmo estrae da Q il vertice u con la stima di distanza minima assoluta (greedy) e lo inserisce nell'insieme dei nodi
3. L'algoritmo esegue il rilassamento di tutti gli archi uscenti da u , per ogni vicino v , controlla se il cammino passante per u è più conveniente della stima attuale v.d. se lo è, aggiorna v.d. e il predecessore v.p.

Dijkstra(G, w, s)

INITIALIZE-SINGLE-SOURCE(G, s) $\rightarrow O(|V|)$

$S = \emptyset$

$Q = G.V$

Mentre $Q \neq \emptyset$ do

$u = \text{EXTRACT-MIN}(Q)$ $\rightarrow O(|V| \log |V|)$

$S = S \cup \{u\}$

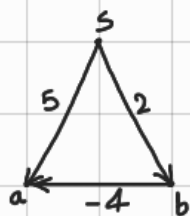
Per ogni vertice $v \in G.Adj[u]$ do

RELAX(u, v, w)

DECREASE-KEY($Q, v, v.d$) $\rightarrow O(|E| \log |V|)$, $|E| \geq |V| - 1 \rightarrow$ Graf connesso

$\text{tot} = O(|E| \log |V| + |V| \log |V|) = O(|E| \log |V|)$

2. Fornire un esempio di grafo diretto e pesato $G = (V, E)$, con $|V| = |E| = 3$ e vertice sorgente s , sul quale l'algoritmo di Dijkstra non opera correttamente, cioè non determina correttamente un cammino minimo da s ad un qualsiasi vertice raggiungibile da s . Si disegni un tale G indicando chiaramente quali siano i pesi e quale il vertice sorgente s . [4 punti]



1. Estrazione di s : La sorgente scopre i vicini e imposta le stime.

$$d[a] = 5, d[b] = 2$$

2. Fallimento greedy: essendo $2 < 5$, l'algoritmo estrae b dalla coda e

crystallizza il suo cammino minimo a 2, poi

rilassa l'arco (b, a) aggiornando il valore di a :

$$d[a] = 2 + (-4) = -2$$

3. Estrazione di a : Viene estratto a con il valore -2 .

ha fallito perché l'algoritmo si basa sull'ipotesi che i cammini estratti siano definitivi e

possono solo crescere. Se dopo l'estrazione di b ci fosse stato un arco uscente da a che

tornava verso b , Dijkstra non lo avrebbe mai riesaminato.